CSED233 · 데이터구조 · 포스텍

# Ch6. Trees (2)

## 1. DFS vs BFS 복습

### 두 순회 방식

- **DFS (Depth-First)**: 깊이 먼저 → **Stack(스택)** 기반 (recursion = call stack)
- **BFS (Breadth-First)**: 같은 level 먼저 → **Queue(큐)** 기반 (level-order)
- 둘 다 $O(n)$ (모든 노드 1회 방문)

### Pseudocode(의사코드)

### DFS (iterative)

```
DFS(root):
    if root == null: return
    stack.push(root)
    while stack not empty:
        cur = stack.pop()
        visit(cur)
        for child of cur in right-to-left order:
            stack.push(child)
```

DFS (iterative, left-to-right preorder): 스택은 마지막에 넣은 노드를 먼저 꺼내므로, 왼쪽 자식부터 방문하려면 현재 노드의 자식들을 오른쪽부터 왼쪽 순서로 스택에 넣어야 한다. 빈 트리에서는 아무 노드도 방문하지 않는다.

### BFS (level-order)

```
BFS(root):
    if root == null: return
    queue.enqueue(root)
    while queue not empty:
        cur = queue.dequeue()
        visit(cur)
```

```
    for each child of cur:
        queue.enqueue(child)
```

빈 트리에서는 아무 노드도 방문하지 않는다. 비어 있지 않은 트리에서는 루트를 먼저 큐에 넣고, 노드를 꺼낼 때마다 방문한 후 자식들을 정해진 순서대로 큐에 넣는다. 큐의 FIFO 특성 때문에 낮은 level부터 차례로 방문한다.

- **핵심 차이**: Stack → LIFO → 깊이 방향 / Queue → FIFO → 너비 방향

## 2. Tree Reconstruction (재구성)

---

### 질문

- 순회 결과만 보고 원래 tree 구조를 재구성할 수 있는가?

### 결론

- **전제**: 순회 결과로부터 이진트리의 유일한 재구성을 보장하려면 일반적으로 모든 노드의 값이 서로 다르다고 가정한다. 중복 값이 있으면 inorder에서 루트 값에 해당하는 위치가 여러 개일 수 있으므로 재구성의 유일성이 보장되지 않는다. 다만 중복 값이 존재하더라도 순회 결과에 따라 트리가 유일하게 결정되는 경우는 있다.
- **단일 순회 불가**: 단일 순회만으로는 일반적으로 불충분하다. 추가적인 구조 제약이 없고 노드 값이 모두 다른 일반 이진트리에서는 preorder, inorder, postorder 중 하나만으로 모든 경우에 트리를 유일하게 재구성할 수 없다. 다만 노드가 하나뿐인 경우처럼 구조가 유일하게 결정되는 특수한 경우가 있으며, 서로 다른 키를 갖는 BST처럼 추가 조건이 주어지면 preorder 또는 postorder 하나만으로도 재구성이 가능하다.
- 추가적인 구조 제약이 없고 모든 노드 값이 서로 다른 일반 이진트리에서는 preorder + inorder, postorder + inorder, level-order + inorder의 조합으로 트리를 유일하게 재구성할 수 있다. Preorder + postorder만으로는 일반적으로 유일 재구성이 불가능하다. 단, full binary tree와 BST처럼 추가 조건이 주어진 경우에는 다른 순회 조합이나 단일 순회만으로도 재구성이 가능하다.
  - ✓ (preorder + inorder)
  - ✓ (postorder + inorder)
  - ✓ (level-order + inorder)
  - ✗ (preorder + postorder): 모호성 남음

Tree Reconstruction의 유일성 조건: 모든 노드의 값이 서로 다르고 추가적인 구조 제약이 없는 일반 이진트리를 가정하면, preorder + inorder, postorder + inorder, level-order + inorder의 조합으로 트리를 유일하게 재구성할 수 있다. 반면 preorder + postorder만으로는 자식이 하나뿐인 노드의 자식이 왼쪽에 있는지 오른쪽에 있는지 결정할 수 없어 일반적으로 유일한 재구성이 불가능하다. 단, 모든 내부 노드가 정확히 두 자식을 갖는 full binary tree에서는 preorder + postorder만으로도 유일하게 재구성할 수 있다. 또한 BST처럼 노드 값의 순서에 관한 추가 조건이 주어진 경우에는 preorder 또는 postorder 하나만으로도 재구성이 가능하다.

**왜 일반 이진트리의 대표적인 재구성 조합에서 inorder를 사용하는가?**

Preorder의 첫 원소, postorder의 마지막 원소, level-order의 첫 원소를 이용하면 해당 트리의 루트를 식별할 수 있다. 여기에 inorder를 결합하면 루트가 나타나는 위치를 기준으로 왼쪽 서브트리와 오른쪽 서브트리의 노드를 구분할 수 있다. 노드 값이 모두 다른 일반 이진트리에서는 이 과정을 재귀적으로 반복하여 트리를 유일하게 재구성할 수 있다. 단, full binary tree나 BST처럼 추가적인 구조 또는 키 순서 조건이 주어지면 inorder 없이도 재구성이 가능한 경우가 있다.

**알고리즘 (Preorder + Inorder)**

Preorder + Inorder 재구성 알고리즘: 노드 값이 모두 다르고 두 순회 결과가 같은 이진트리에서 생성되었다고 가정한다. Preorder의 첫 원소를 루트로 선택하고, inorder에서 루트가 나타나는 위치를 기준으로 왼쪽·오른쪽 서브트리를 구분한다. 왼쪽 서브트리의 노드 수만큼 preorder에서도 왼쪽 부분을 분리하고 나머지를 오른쪽 부분으로 사용한다. 각 부분에 대해 동일한 과정을 재귀적으로 반복하며, 부분 배열이 비어 있으면 null을 반환한다.

```
reconstruct(pre, ino):
    if length(pre) == 0: return null
    root = Node(pre[0])
    j = indexOf(ino, pre[0])
    leftIn = ino[0:j], rightIn = ino[j+1:]
    leftPre = pre[1:1+j], rightPre = pre[1+j:]
    root.left = reconstruct(leftPre, leftIn)
    root.right = reconstruct(rightPre, rightIn)
    return root
```

여기서 *j*는 왼쪽 서브트리의 노드 수이다. 각 재귀 호출에서도 preorder와 inorder가 동일한 서브트리의 노드들을 나타내도록 배열을 분할하는 것이 핵심이다.

**예시**

- Inorder(중위 순회): ⟨20, 30, 35, 40, 45, 50, 55, 60, 70⟩

- Postorder(후위 순회): ⟨20, 35, 30, 45, 40, 55, 70, 60, 50⟩
- → Root = 50 (postorder 끝)
- → Left: {20,30,35,40,45}, Right: {55,60,70}
- → 재귀 반복하여 재구성

## 3. General k-ary Tree

---

#### 정의

- **k-ary tree**: 각 노드의 자식 수가 최대 $k$개
- **일반 tree**: 자식 수 제한 없음, 순서 없을 수도 있음
- Binary tree도 각 노드의 실제 자식 수는 0~2개로 가변적이다. 차이는 Binary tree가 왼쪽·오른쪽의 두 자식 위치를 구분하고 자식 수를 최대 2개로 제한한다는 점이다. 반면 k-ary tree는 노드당 최대 k개의 자식을 허용하며, 일반 트리는 특정한 고정 상한 k를 반드시 전제하지 않는다. 이처럼 일반 트리는 자식 관계를 표현하기 위해 Parent-Pointer, List-of-Children, LC/RS 등의 다양한 구현 방식을 사용할 수 있다.

#### 대표 구현 3가지

1. **Parent-pointer array** (simple approach)
2. **List-of-Children**
3. **Left-Child / Right-Sibling**

   각각 pros/cons 있음 → 연산 빈도에 따라 선택.

## 4. Simple Approach (Parent-Pointer)

---

#### 구조

- 두 병렬 배열:
  - `Tree[k] = m` → 노드 $k$의 parent가 노드 $m$
  - `Label[k]` → 노드 $k$의 value
- Root는 `Tree[root] = 0` (convention)

**시간복잡도**

| 연산 | 복잡도 | 이유 |
|---|---|---|
| `Parent(k, T)` | $O(1)$ | `Tree[k]` 한번 접근 |
| `Leftmost_Child(k, T)` | $O(n)$ | 전체 배열 스캔 필요 (children 정보 없음) |
| `Create(v, T₁,...,Tₘ)` | $O(n)$ | Create(v,T1,...,Tm): 독립된 서브트리 배열들을 하나의 연속 배열로 통합해야 하는 구현에서는 노드 복사 및 필요한 인덱스·parent 정보 갱신으로 전체 노드 수 n에 비례하는 O(n) 시간이 들 수 있다. 반면 모든 서브트리가 이미 동일한 인덱스 공간에 충돌 없이 저장되어 있다면 기존 노드의 재배치는 필요하지 않으며, 새 루트를 생성하고 m개 서브트리 루트의 parent 정보를 갱신하는 데 O(m)이 든다. 특히 m=2로 고정된 C2 연산의 링크 갱신은 O(1)이다. |

**Create 연산의 어려움**

- 독립된 두 트리가 각각 별도의 인덱스 배열에 저장되어 있고, 이들을 새 루트 아래 하나의 연속 배열로 통합해야 하는 구현에서는 인덱스 충돌을 방지하기 위해 두 번째 트리의 노드 인덱스를 이동하고 관련 parent 정보도 갱신해야 한다. 반면 모든 노드가 이미 동일한 인덱스 공간에서 충돌 없이 관리되고 새 루트를 위한 공간도 확보되어 있다면, 기존 노드의 인덱스 이동 없이 새 루트 생성과 기존 루트들의 parent 정보 갱신만으로 트리를 결합할 수 있다.
- 예: $T_1$이 6 노드, $T_2$가 3 노드 → $T_2$의 index 1,2,3 → 8,9,10으로 변경 + parent 값도 전부 수정
- → $O(n)$ 작업
- Create의 $O(n)$은 독립 배열을 하나의 연속 배열로 통합하는 구현에서의 비용이며, 동일 배열에서 고유 인덱스로 관리되면 재배치가 필수가 아니다.

## 5. List-of-Children Approach

**아이디어**

- Parent info **+ children info 동시 유지**

- 각 non-terminal 노드에 대해 **linked list of children** 추가

**Combining Trees:** $C_2(v, T_1, T_2)$

- **C2의 정의**: $C_2(v,T_1,T_2)$는 값이 $v$인 새 루트를 만들고, $T_1$의 루트를 첫 번째 자식으로, $T_2$의 루트를 두 번째 자식으로 연결한 트리를 반환한다.
- **아래첨자 2의 의미**: 숫자 2는 결합하는 기존 트리가 두 개라는 뜻이며, 결과 트리가 반드시 binary tree여야 한다는 뜻은 아니다.
- **일반 Create와의 관계**: Create$(v,T_1,\ldots,T_m)$는 $m$개의 기존 트리를 새 루트 아래에 결합하는 일반 연산이다. 따라서 $C_2(v,T_1,T_2)$는 $m$=2인 특수한 경우이고, $C_2(v,T_1,T_2)$ = Create$(v,T_1,T_2)$이다.
- List-of-Children에서의 $C_2$ 구현: 두 트리 $T_1$, $T_2$가 이미 동일한 노드 배열에 충돌 없는 인덱스로 저장되어 있고 새 루트 노드를 상수 시간에 생성할 수 있다면, 새 루트 $v$의 children list에 $T_1$과 $T_2$의 루트를 순서대로 연결하고 두 기존 루트의 parent 필드를 $v$로 변경하면 된다. 연결할 기존 루트가 두 개이므로 필요한 리스트 연결 및 parent 갱신은 상수 번 수행되며, $C_2$의 시간복잡도는 $O(1)$이다. 이 결론은 결과 트리가 binary tree인 경우뿐 아니라 두 트리를 결합하는 일반 트리 연산에도 적용된다.
- 분리 배열 저장 시의 비용: $T_1$과 $T_2$가 서로 다른 연속 인덱스 배열에 저장되어 있고 결과 트리를 하나의 연속 배열로 통합해야 한다면, 노드 복사와 인덱스 및 parent 정보 갱신이 추가로 필요하다. 전체 노드 수를 $n$이라 할 때 이 과정의 시간복잡도는 $O(n)$이다. 따라서 $C_2$ 연산 자체의 링크 갱신 비용은 $O(1)$ 이지만, 실제 전체 수행 시간은 노드 저장 방식과 배열 통합 필요 여부에 따라 달라진다.

**문제점**

1. **Right sibling 접근 어려움**: $y$의 sibling을 찾으려면 → parent 찾고 → parent의 children list 뒤져야 함
2. - **데이터 저장**: List-of-Children에서는 각 노드의 실제 데이터 레코드를 node array에 한 번만 저장한다. 부모 노드의 children list에는 자식 노드의 인덱스 또는 포인터를 저장하므로, 노드 데이터 전체가 복제되지는 않는다.
   - **추가 공간의 원인**: 각 부모-자식 관계를 표현하기 위한 리스트 항목과 연결 포인터가 추가로 필요하다.
   - **공간복잡도**: $n$개 노드를 가진 하나의 일반 트리에는 $n-1$개의 간선이 있다. 간선마다 상수 크기의 리스트 항목을 하나씩 저장하는 구현에서는 추가 공간이 $O(n)$이다.

## 6. Left-Child / Right-Sibling (LC/RS)

---

### 아이디어

- 각 노드에 정확히 **2개의 pointer**만 저장:
  - `Lc` : leftmost child
  - `Rs` : right sibling
- 추가로 `Pa` (parent)도 저장 가능
- **자식 수 무관** → 어떤 $k$-ary tree든 2-pointer로 표현

### 핵심 통찰

**어떤 general k-ary tree도 binary tree로 변환 가능!**

- **변환의 전제**: 각 노드의 자식 순서가 정해진 rooted general tree는 LC/RS 표현을 이용하여 이진트리 형태로 정보 손실 없이 변환할 수 있다. 자식 순서가 없는 일반 트리는 변환 전에 자식 순서를 하나 정해야 한다.
- **링크의 의미**: 원래 트리의 첫 번째 자식은 변환된 이진트리의 left 링크로, 바로 오른쪽 형제는 right 링크로 표현한다.
- **대응 관계**: 이 변환은 일부 부모-자식 관계를 형제 링크로 인코딩하므로, 원래 일반 트리와 변환된 이진트리가 그래프 동형이라는 뜻은 아니다. 정확히는 순서가 있는 일반 트리와 루트의 right 링크가 null인 LC/RS 이진트리 표현 사이에 일대일 대응이 존재한다.
- **Forest로의 확장**: 여러 일반 트리의 루트를 형제처럼 연결하면 forest 역시 하나의 이진트리 형태로 인코딩할 수 있다.

### Dynamic (Linked) 구현

- array 고정 크기 문제 → linked list로:

```
struct node {
   char label;
   node* lc;   // leftmost child (= binary's left)
   node* rs;   // right sibling  (= binary's right)
};
```

- → **사실상 binary tree node 구조**

## 7. Knuth Transform (변환 절차)

**3단계**

1. **Leftmost child → Left child**
2. **Right sibling → Right child** (단, tree root들은 서로 sibling으로 간주 — forest 처리)
3. **시계방향 45° 회전** → 시각적으로 binary tree 형태 완성

**순회 결과 비교**

| 순회 | Original general tree | Converted binary tree |
|---|---|---|
| Preorder(전위 순회) | `a b d e f c` | `a b d e f c` ✓ **동일** |
| Inorder | `d b e f a c` | `d e f b c a` ✗ 다름 |
| Postorder | `d e f b c a` | `f e d c b a` ✗ 다름 |

- General Tree의 Inorder 규약: 일반 트리에는 이진트리와 달리 보편적으로 통일된 inorder 정의가 없다. 이 표의 Original general tree 열에서는 '첫 번째 자식 서브트리를 재귀적으로 inorder 순회한 후 현재 루트를 방문하고, 나머지 자식 서브트리들을 왼쪽부터 차례대로 inorder 순회한다'는 규약을 사용한다. Converted binary tree 열에는 일반적인 이진트리 inorder, 즉 left subtree → root → right subtree 순서를 적용한다. 따라서 표의 두 inorder 결과가 서로 다른 것은 정상이며, 이 비교는 명시한 일반 트리 inorder 규약에 따른 결과임에 주의해야 한다.
- **핵심**: 자식 순서가 정해진 일반 트리의 preorder는 LC/RS 변환 전후에 동일하다. 일반 트리의 preorder는 현재 노드를 먼저 방문하고 자식 서브트리들을 왼쪽부터 순서대로 방문한다. LC/RS에서는 첫 번째 자식이 left 링크로, 다음 형제가 right 링크로 연결되므로, 변환된 이진트리의 root → left → right 순회가 원래 일반 트리의 방문 순서를 그대로 재현한다.

### Q1. Converted binary tree를 general tree로 역변환하려면?

> **힌트**

LCRS 링크의 의미를 뒤집는다: left = 첫째 자식, right = 다음 형제.

> **정답**

개념은 힌트 참고.

반대 과정. root의 left child → leftmost child, 그 노드의 right chain → sibling들. 시계방향 반대로 45° 회전.

## 8. 구현 비교 정리

### 시간복잡도 요약

전제: $n$은 전체 노드 수, deg($p$)는 부모 $p$의 자식 수이며, 직접 접근을 가정한다. Parent-Pointer의 형제 순서는 배열 인덱스 오름차순을 가정한다.

| 연산 | Simple (Parent) | List-of-Children | LC/RS |
|---|---|---|---|
| Parent(k) | $O(1)$ | $O(1)$, parent 저장 | $O(1)$, Pa 저장 시 |
| Leftmost child | $O(n)$ | $O(1)$, head 접근 | $O(1)$ |
| Right sibling | $O(n)$ | $O(\deg(p))$ | $O(1)$ |

LC/RS가 Lc와 Rs만 저장하고 Pa를 저장하지 않는 경우, Parent(k)는 트리 탐색이 필요하여 최악 $O(n)$이다. 반면 Pa를 추가로 저장하면 Parent(k)는 $O(1)$이지만 노드당 구조 포인터가 세 개 필요하다. List-of-Children에서 Right sibling 조회는 부모의 자식 리스트를 순차 탐색하는 구현을 가정한 최악 시간복잡도이며, deg($p$)는 해당 부모의 자식 수를 뜻한다.

| 항목 | Simple (Parent) | List-of-Children | LC/RS |
|---|---|---|---|
| `Create(v, T₁,...,Tₘ)`, 가변 m | 동일 인덱스 공간에서는 $O(m)$, 연속 배열 통합이 필요하면 $O(n)$ | 재배치 없이 연결하면 $O(m)$, 별도 연속 배열 통합 시 $O(n)$ | 기존 노드를 이동하지 않고 링크를 연결하면 $O(m)$ |
| $C_2(v,T_1,T_2)$, 고정 $m=2$ | 재배치 없이 parent만 갱신하면 $O(1)$, 배열 통합 시 $O(n)$ | 재배치 없이 리스트와 parent를 갱신하면 $O(1)$, 별도 연속 배열 통합 시 $O(n)$ | 기존 두 루트를 링크로 연결하면 $O(1)$ |
| 공간 특성 | 부모 인덱스 저장 | 자식 리스트 항목 추가 | 고정 개수 포인터 |

Create(v,T1,...,Tm)의 시간복잡도는 결합할 기존 트리의 개수 m과 노드 저장 방식에 따라 결정된다. 일반적인 Create에서는 m이 가변적이므로 재배치 없이 연결하더라도 O(m) 시간이 필요하다. C2(v,T1,T2)는 m=2로 고정된 특수한 경우이므로 노드 재배치가 불필요하면 O(1)에 처리할 수 있다. 단, 별도의 연속 배열을 하나로 통합해야 하는 구현에서는 추가적인 O(n) 비용이 발생할 수 있다.

**선택 기준**

- **Parent 조회 위주** → Simple
- **Children 순회 위주** → List-of-Children
- **일반 tree를 binary tree처럼 다루고 싶다** → LC/RS (+ Knuth transform)

## Summary

---

- DFS = stack, BFS = queue, 둘 다 $O(n)$
- 재구성: 노드 값이 모두 다른 일반 이진트리는 inorder + (preorder/postorder/level-order)로 유일 재구성 가능. 예외적으로 full binary tree는 preorder + postorder, BST는 preorder 또는 postorder 하나만으로도 유일 재구성 가능.
- General tree 구현 3가지: Parent / List-of-Children / LC-RS
- **Knuth transform**: general tree → binary tree (preorder 보존)